3D Plan Visualization: Automated Building Reconstruction and Walkthrough Generator

Design Document

Team Number: sdmay27-14

Client: Buildertrend

Advisers: Dr. Judith Islam

Team Members/Roles:

Nakshatra Gupta - Communication Lead

Rishiraj Parmar - Organization Lead

Ryan McNab - Team Meetings Lead

Daniel Hargrave - Computer Vision Lead

Jase Beaubien - Graphics Lead

Team Email

Team Website: https://sdmay27-14.sd.ece.iastate.edu/

Revised: 10th Nov 2026

Executive Summary

Provide a brief (300-500 word) summary of your project. This summary should give the reader enough information to understand the purpose, design strategies, and project progress thus far. It should address at least the following questions/items:

- What is the problem and why is it important?
- What are your key design requirements?
- Summarize your design, include approaches and technologies used.
- How much progress have you made thus far?
- How well does your design meet requirements and address user needs? How do you know?
- What are potential next steps to be taken?

Construction plans are usually shared as 2D floor plans and elevation drawings in PDF, CAD, or paper form. Homeowners, subcontractors, and other non-technical stakeholders find it hard to picture the finished building from them. This leads to miscommunication, change orders, and costly rework. Our client, Buildertrend, already stores plans and project documents on its construction management platform, so turning those existing documents into a 3D experience would help users understand a project without extra work.

The pipeline must accept at least one common plan format (PDF, DWG/DXF, or scanned images) and extract walls, room boundaries, and dimensions from floor plans. It must derive vertical information such as wall heights, roof pitch, and window and door placement from elevations and reconcile it with the floor plan. It must produce a dimensionally consistent 3D model, show it in a browser viewer with orbit controls, and support a first-person walkthrough, which the team treats as a required feature. It must also degrade gracefully on imperfect or hand-drawn input.

The team is split into a Computer Vision team and a Graphics team. The Computer Vision side, in Python, reads plans and outputs structured data. The Graphics side takes that data and builds and displays the 3D model and the walkthrough. A shared data format, currently a draft JSON schema, connects the two sides so each can be developed and tested independently. The team plans to focus this semester on single-floor plans, and multi-story plans will be covered next semester. Early prototype work includes a first demo that turns a floor plan image into an editable 3D model with a walk mode.

Team roles, policies, and the two-team structure are in place, and the team has met with Buildertrend and received sample plans. A team website is live and a prototype is built. On the data side, one sample plan was analyzed and found to be a CAD-exported PDF with a readable text layer. A Python extraction script read its sheets, door and window schedules, dimensions, and roof pitches into JSON, and the counts matched across the schedules and the floor plan tags

It is early, and the evidence is limited. The script worked on the first sample plan, which suggests that for CAD-exported PDFs much of the data can be read directly. It did not work on the three other sample plans, so it does not yet handle the variety of real plans. The 3D generation, the walkthrough, and handling of poor input have not been tested against ground-truth measurements. The sample plan also showed features the draft schema does not cover, such as multiple roof pitches and porches. Features that depend on multiple floors, such as open-to-below spaces, can wait until next semester.

Learning Summary

Development Standards & Practices Used

List all standard circuit, hardware, software practices used in this project. List all the engineering standards that apply to this project that were considered.

Summary of Requirements

List all requirements as bullet points in brief.

Applicable Courses from Iowa State University Curriculum

List all Iowa State University courses whose contents were applicable to your project.

New Skills/Knowledge Acquired

List all new skills/knowledge that your team acquired which was not part of your Iowa State curriculum in order to complete this project.

Table of Contents

1 Introduction 7
- 1.1 Problem Statement 7
- 1.2 Intended Users 7

2 Requirements, Constraints, and Standards 7
- 2.1 Requirements & Constraints 7
- 2.2 Engineering Standards 7

3 Project Plan 8
- 3.1 Project Management and Tracking 8
- 3.2 Task Decomposition 8
- 3.3 Project Proposed Milestones, Metrics, and Evaluation Criteria 9
- 3.4 Project Timeline and Schedule 9
- 3.5 Risks and Risk Management/Mitigation 9
- 3.6 Personnel Effort Requirements 10
- 3.7 Other Resource Requirements 10

4 Design 10
- 4.1 Design Context 10
  - 4.1.1 Broader Context 10
  - 4.1.2 Prior Work/Solutions 11
  - 4.1.3 Technical Complexity 11
- 4.2 Design Exploration 11
  - 4.2.1 Design Decisions 11
  - 4.2.2 Ideation 11
  - 4.2.3 Decision-Making and Trade-Offs 11
- 4.3 Proposed Design 12
  - 4.3.1 Overview 12
  - 4.3.2 Detailed Design and Visual(s) 12
  - 4.3.3 Functionality 12
  - 4.3.4 Areas of Concern and Development 12
- 4.4 Technology Considerations 12
- 4.5 Design Analysis 13

5 Testing 13
- 5.1 Unit Testing 13
- 5.2 Interface Testing 13
- 5.3 Integration Testing 13
- 5.4 System Testing 13
- 5.5 Regression Testing 14
- 5.6 Acceptance Testing 14
- 5.7 Security Testing (if applicable) 14
- 5.8 User Testing 14
- 5.9 Testing Results 14

6 Implementation 14

7 Ethics and Professional Responsibility 14
- 7.1 Areas of Professional Responsibility/Codes of Ethics 15
- 7.2 Four Principles 15

7.3 Virtues 15
8 Closing Material 16
8.1 Conclusion 16
8.2 References 16
8.3 Appendices 16
9 Team 16
9.1 Team Members 16
9.2 Required Skill Sets for Your Project 16
9.3 Skill Sets Covered by the Team 16
9.4 Project Management Style Adopted by the team 16
9.5 Initial Project Management Roles 17
9.6 Team Contract 17

List of figures/tables/symbols/definitions

Figure 1. Empathy Map 6

Figure 2. Gantt Chart 13

Figure 3. Block Diagram Overview 20

Table 1. Risk Management Table 14

Table 2. Estimated Time Table 14/15

Table 3. Broader Context Table 16

Table 4. Decision Making Table 19

1 Introduction

1.1 Problem Statement

Homeowners, subcontractors, and non-technical stakeholders working with Buildertrend tend to struggle to make a mental model from their designs effectively on a piece of paper. They need some platform to translate their 2D visualizations into actual interactive visual models. It matters because this disconnect drives misunderstandings, which then result in changes of costs, avoidable rework, and delays in planning and building. This can directly affect the relationship between the homeowners and contractors. To address this problem, we will build a model that converts 2D floor plans into dimensionally consistent 3D visualizations with interactive features.

1.2 Intended Users

3 users that we thought of for this project are future homeowners, Buildertrend designers and employees in sales.

For the future homeowners, we need to create a complete visual model of their provided design document so that they are able to see an accurate model of the building that they will be potentially building/remodeling. This benefits them by avoiding misunderstanding and there aren't avoidable reworks later in the process.

For the designers at Buildertrend, we need to ensure that our design lines up exactly with the blueprints. This will avoid the customers misunderstanding something from the document and asking for rework on a thing that is already good. In the figures section there is an empathy map that describes this group more in detail.

For the people in sales, we need to create an easy to use software platform so that they do not need to have much knowledge in software or 3d modelling to understand how to use the platform and describe how it works on a basic level. This project will help sales because they won't have to have any technical experience to communicate the build plans properly. Through a visual representation, they will have an easier time conveying it.

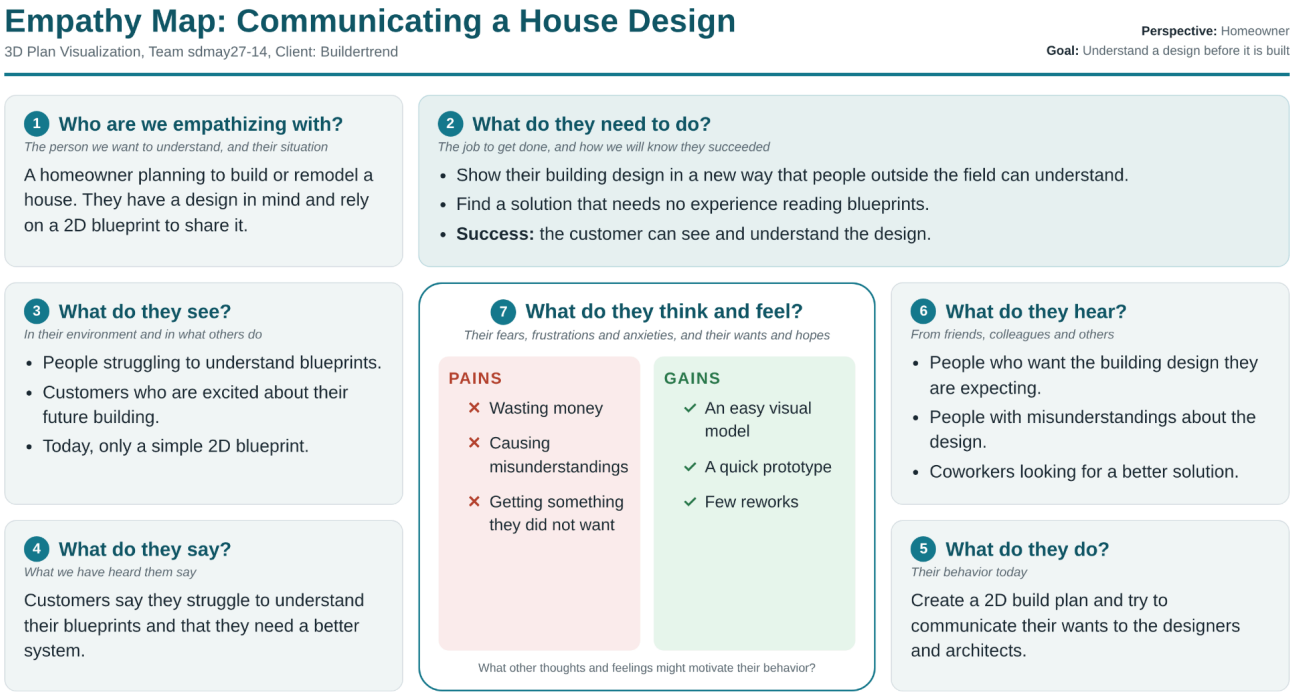


Figure 1. Empathy map

2 Requirements, Constraints, and Standards

2.1 Requirements & Constraints

FR-1: The system should accept plan sets as PDF files. CAD-exported PDFs with a text layer are the primary format, and scanned images (PNG/JPG) are a secondary format.

FR-2: The system should extract exterior and interior walls, room boundaries, room labels, and dimension strings from floor plan sheets.

FR-3: The system should extract door and window schedules and match each scheduled item to its tag on the floor plan.

FR-4: The system should extract vertical information from elevation sheets, including plate/wall heights, roof pitches, and window and door head and sill heights.

FR-5: The system should reconcile floor plan and elevation data into one model. When the two sources conflict, it shall flag the conflict instead of silently choosing one value.

FR-6: The system should output extracted plan data as JSON that validates against the team's versioned schema. This schema is the contract between the Computer Vision and Graphics subsystems.

FR-7: The system should generate a 3D model from the JSON with walls, openings, floor, and roof, keeping dimensions consistent with the source plan.

FR-8: The system should display the model in a web browser or desktop application with orbit, pan, and zoom controls.

FR-9: The system should provide a first-person walkthrough mode with keyboard/mouse movement and collision with walls.

FR-10: The system should degrade gracefully on imperfect or hand-drawn input. It shall produce a partial model, mark low-confidence elements, and report what it could not extract instead of failing outright.

FR-11: The system should let the user manually correct misidentified elements (for example, move a wall or change a door width) before or after 3D generation.

PR-1: On CAD-exported PDFs, door and window counts shall match the schedules 100% of the time. On scanned plans, detection shall find all of the doors and windows. (constraint)

UI-1: A non-technical user (homeowner or salesperson) shall be able to go from uploading a plan to viewing the 3D model without training and in no more than three main steps. (constraint)

UI-2: Room names from the plan shall be shown as labels in the 3D view, and users shall be able to toggle them on and off.

UI-3: Elements flagged as low-confidence (FR-10) shall be visually distinguished so users know which parts of the model to double-check.

AR-1: The model shall use simple, neutral materials with clearly distinguishable walls, floors, roof, doors, and windows. Realistic texturing is not required.

AR-2: The model shall look like the building in the plans when compared side by side with the elevation drawings, including roof shape, window layout, and overall proportions.

RR-1: The system shall not require a dedicated GPU for processing or viewing. (constraint)

RR-2: The design shall be able to integrate with Buildertrend's existing document storage so users don't have to re-upload or re-draw plans.

SR-1: Client plan files shall be treated as confidential and shall not be sent to third-party services, including external AI APIs, without Buildertrend's approval.

SR-2: Sample plans provided by Buildertrend shall be stored only in team-controlled, access-restricted locations and shall not be published on the team website or in public repositories.

MR-1: The most important parts of the project will be heavily tested through unit testing to ensure the code is maintainable.

MR-2: Code shall be kept in a shared Git repository with documented setup instructions so another team or Buildertrend engineers can run it.

2.2 Engineering Standards

Q1) Browse the videos and interactive pages available at the IEEE Standards website: https://standards.ieee.org/about/standards-in-action/

We looked at "What are Standards and where are they?", the Interactive Cityscape, and Standards in Healthcare on the IEEE Standards in Action site. Together they show that standards are agreed-upon ways of doing things that let products from different companies work together safely and reliably, like a phone pairing with a speaker and a car, or health devices sharing data with different apps.

Standards matter for our project because our system has to read plan files made by other software, like PDFs and DWG/DXF files, and produce 3D models that can work inside Buildertrend's platform. Our two subteams also rely on a shared JSON format, which acts as our own internal standard. Without these common rules, the parts of our system wouldn't fit together.

Q2) Browse the IEEE Standards website, and choose 3 standards under a category.

Category: Quality management and quality assurance

Standard 1: Quality management — Guidelines for configuration management

ISO 10007 explains how to manage a product's configuration, which includes planning, identifying configuration items, controlling changes, tracking their status, and auditing. It is about knowing exactly which version of everything you have and controlling how it changes.

Standard 2: Quality management — Guidelines for quality management in projects

ISO 10006 gives guidelines for applying quality management to projects of any size or length, whatever the product or service. It covers how to build quality into project management processes, and it includes guidance on writing a quality plan for the project.

Standard 3: Quality management systems — Requirements

ISO 9001 sets the requirements for a quality management system, specifically on how an organization plans its work, checks results, responds to problems, and keeps improving.

Q3) After reviewing some of the technical details of the three selected standards, do you believe it has relevance to your project? Why or why not? Be specific.

These technical standards do all have relevancy to our project. First off, for standard 1 it is important to our project because our two subteams depend on a shared JSON schema. If the Computer Vision team changes the schema without the Graphics team knowing, the system breaks. We plan to apply this standard in three ways:

- Version the schema.
- Require both subteams to agree before any schema change, such as adding porches or multiple roof pitches.
- Use Git to track every version of the code, schema, and test plans, so we can always tell which version produced a given result.

Standard 2 is important to our project because ours is only a two-semester project with a client, which is the situation this standard was written for. We can use its guidance to write a simple quality plan. The plan would cover who reviews each deliverable, how we check the JSON output and 3D model against our requirements before each client meeting, and how we track and fix problems found in testing.

Standard 3 is important to our project because its 3 main ideas directly relate to our project. First, the idea of customer focus means building around what Buildertrend and its users actually need. Next, evidence based decisions mean judging how the project works by measuring its accuracy against the sample plans, not by whether the final design looks right. Continual improvement is important to our project because throughout the project we will likely be receiving plans that fail during the process. We will need to modify the model each time this happens.

Q4) Review with your team the standards that each of you have selected. What other standards did some of your team members choose that are different?

While some of our team focused on quality management standards (ISO 10007, 10006, and 9001),

Rishiraj looked at ISO/IEC/IEEE 14764 (Software Maintenance), which describes how to plan and manage changes after software is delivered, which fits our project since it spans two semesters and our schema and extraction script will keep changing. IEEE 1028 (Software Reviews and Audits) defines procedures for reviews like inspections and walkthroughs, which helps our two subteams catch mismatches early. ISO/IEC/IEEE 29119 (Software Testing) gives a consistent way to plan and document tests, which we need because real plans vary so much. Their standards cover quality at the organization and project level, while his covers specific software practices, so together they support each other.

Ryan looked at ISO/IEC 12113:2022, which is called Information technology. This standard defines glTF, a file format for delivering 3D models to applications. glTF is not tied to any particular graphics API. Its purpose is to connect the tools that create 3D content with the applications that display it, using a format that is efficient, extensible, and interoperable for sending and loading 3D content. This standard is important to our project because the Graphics team has to turn the extracted floor plan data into a 3D model that loads quickly in a web browser. glTF is designed for exactly that use, and the common web 3D libraries support it directly. Generating our models as glTF files means:

- The models will display the same way in any modern browser.
- Users can download the model and open it in other 3D tools.
- Buildertrend could later bring the models into its own platform without a custom converter.

Jase looked at IEEE 754-2019 for floating point arithmetic, since the bindings between the central and graphics processing units need to function under the same digital standard. The shaders use floats extensively due to their use of vertices and matrices. Enforcing this standard is critical since the CPU and GPU are two separate processing units. Misaligning buffer data to the GPU, even by a small amount, causes critical rendering problems.

Daniel looked at IEEE 3110-2025 which defines the technical requirements and functional specifications for APIs that describe how computer vision algorithms connect to elements in deep learning. It handles guidelines for how CV models handle input tensors, parameters, and execution workflows. The standard enables developers to build modular, interoperable machine learning pipelines.

Nakshatra looked at ISO 32000-2:2020, which defines the structure of PDF 2.0 files, including how text, fonts, vector graphics, and page content are stored so documents stay readable and consistent across software and devices. This standard is important to our project because the Computer Vision subsystem has to read CAD-exported PDFs as its primary input (FR-1). Since these files store text and vector paths directly, the extraction script can pull wall geometry, room labels, door and window tags, and dimension strings at their exact coordinates instead of guessing from pixels, which supports dimensional accuracy. It also explains why scanned plans, which contain only an image and no text layer, are a harder and lower-confidence input path (FR-10), and files that do not follow the standard can be detected and handled as lower-confidence input. Our quality management standards describe how the team manages its work, while this one defines the format of the input data the whole pipeline depends on, so together they support each other.

Q5) In what specific ways do you plan to incorporate these standards into the design and implementation of your project?

- **Configuration management (ISO 10007):** The shared Git repository (MR-2) will serve as the control point. Configuration items will be identified up front: the JSON schema, the extraction code, the 3D generation code, and the sample plans. The schema will use semantic version numbers, and any change to it will require a documented change request reviewed by both subteams before merging. Tagged releases will mark the version demonstrated at each client meeting. Sample plans will be kept in a team-controlled, access-restricted location (SR-2), with a log of which plan versions were used for each test run.
- **Project quality plan (ISO 10006):** A short quality plan will define reviewers for each deliverable, a pre-meeting checklist that runs the pipeline on the sample plans and compares output to the requirements, and a defect log recording each problem found, its owner, and its resolution. Code review through pull requests will be required before merging into the main branch.
- **Quality principles (ISO 9001):** Accuracy will be measured against the sample plans using defined metrics, such as door and window count match rate (PR-1) and percent deviation in wall lengths between plan and model, and results will be recorded for each iteration. When extraction fails on a new plan, the cause and the fix will be logged as a corrective action, and the failing plan will be added to the regression test set so the issue does not return. Regular feedback from Buildertrend will be recorded and used to adjust priorities, reflecting customer focus.
- **Information technology (ISO/IEC 12113:2022)** We will be using glTF for our 3d model saves, so that we can serialize and deserialize the models in and out of our app. This will allow us to come back to 3d models created previously and will keep us all on the same track with formatting.
- **Computer vision IEEE 3110-2025** Computer vision is just shy of half of the project. This will be useful for things like feature extraction, object detection, and layout parsing for this project.

3 Project Plan

3.1 Project Management and Tracking

For this project we chose to adopt a hybrid project management style that is a combination of waterfall and agile. This is for a couple reasons. The biggest reason is that during the beginning of our project, there was little information to go off of, and therefore, we spent most of our time working on a prototype and discussing options for models. This prototype was made with a waterfall methodology, and once we started to get more information, we worked on it iteratively, through agile methodologies. We then applied this same format for the real project, getting a base pipeline down before splitting up the work and iterating on the design. There are some very clear goals for our project at the end of the first semester and at the end of the class. In terms of detailed design, our client has given us the ability to modify our scope for the project. This in part depends on what we find is feasible at different points, allowing us to work in minor sprints. This goes for how we have researched, created the initial setup, and developed the individual parts of the project, like the model, graphics, or JSON schema.

For project management, we have used a few different tools in order to track progress throughout the course. Largely we have relied on documenting what each person has done for the week and constant communication through Discord. Through these methods we are able to have a clear understanding of the progress that has been made by each member of the team week by week. Instead of using formal issue tracking, we instead track the work based on actual results each person produces each week, and in some portions, we have used Git to track the work done.

3.2 Task Decomposition

Due to us using a hybrid approach and some of the details regarding this project, some requirements will be modified throughout the progression of the project. We split our project up into a main pipeline and multiple sub-pipelines. The main pipeline is composed of the following:

1) Takes in images of floor plans in the form of several different file types, for example, PNG/JPG. Throws an error and exit gracefully if the image is not what is expected.
2) Apply preprocessing steps to the images to make them more clean for the model to read easier.
3) Uses a computer vision model that takes in the pre-processed images to detect lines and creates labels that represent different elements of the image, like walls, doors, windows, and stairs.
4) Use post-processing steps to correct/improve the labels as well as transform the labels into a designated JSON structure.
5) Allow the user to edit the detected structure by showing the built 2D building alongside the original input image.
6) Take the JSON structure and turn it into a 3D rendering and graphical representation of the building.

3.3 Project Proposed Milestones, Metrics, and Evaluation Criteria

We track our progress with weekly milestones. Each milestone is a checkbox that we mark when the work is done, and the weekly reports record the evidence. The milestones follow the pipeline in Section 3.2: input handling, computer vision detection, post-processing into JSON, the 3D model, and the viewer.

Week 1 (8/25 to 9/22):

- [x] ~~Team roles and policies set~~
- [x] ~~Team split into Computer Vision and Graphics~~

Week 2 (9/23 to 9/29):

- [x] ~~First draft of the shared JSON schema and a validating data model~~
- [x] ~~Prototype with edit mode and a walk mode~~

Week 3 (9/30 to 10/6):

- [x] ~~Schema revised to add stairs, scale and roof styles~~
- [x] ~~More example houses in the prototype~~

Week 4 (10/7 to 10/13):

- [x] ~~First client meeting and sample plans received~~
- [x] ~~Extraction script on the first plan, with door and window counts matching the schedules~~
- [x] ~~Prototype handles second floors and roof types~~

Week 5 (10/14 to 10/20):

- [x] ~~Extraction script works on all four sample plans~~
- [x] ~~Plan to scale back the project made~~

Week 6 (10/21 to 10/27):

- [x] ~~Scope agreed with Buildertrend and the TA~~
- [x] ~~First JSON schema shared with the team~~
- [x] ~~YOLOv11 approved and training started~~

Week 7 (10/27 to 11/3):

- [x] ~~First non-prototype YOLO model that tracks walls, doors, windows and stairs, with confidence levels, tested on Buildertrend plans~~
- [x] ~~Schema v0.2 and datasets of more than 6,000 houses~~
- [x] ~~The prototype returns output in the shared schema~~

Week 8 (11/4 to 11/10):

- [ ] Segmentation model tested, then a model with a rotation (theta) value started
- [ ] Lightning talk and design document v2 worked on

3.4 Project Timeline and Schedule

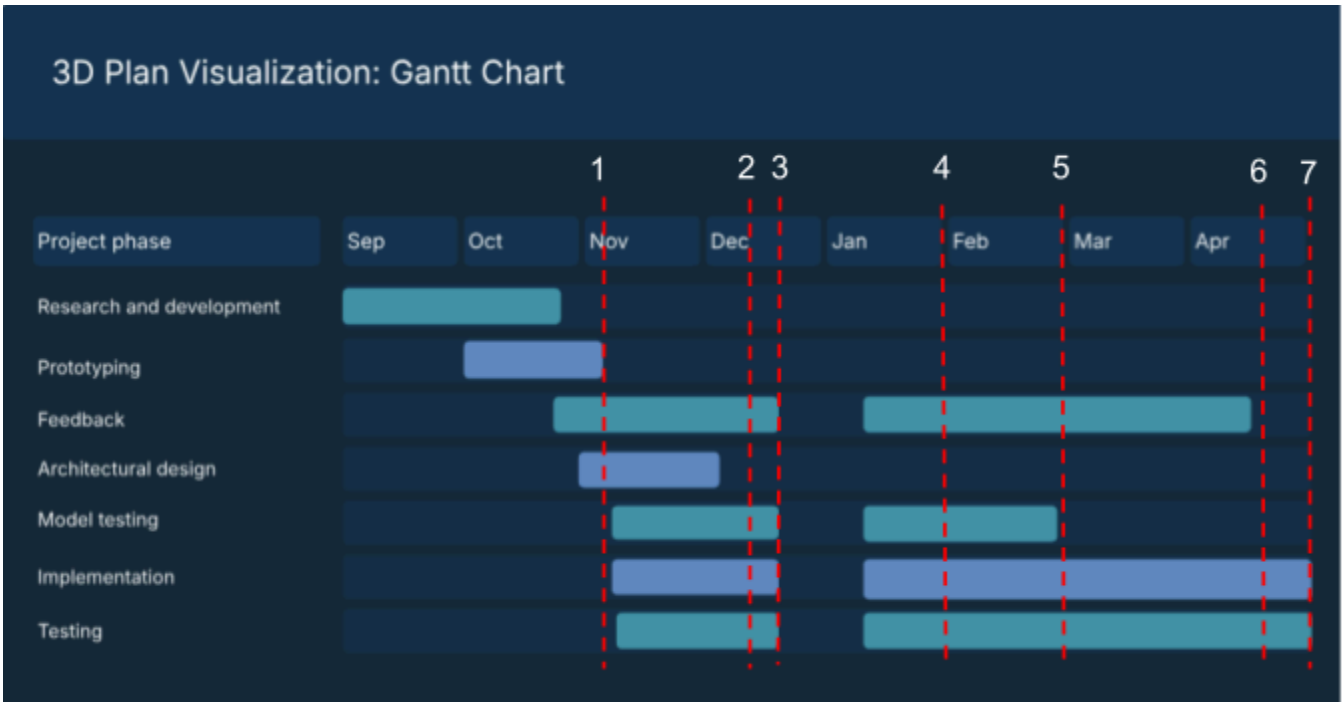


Figure 2. Gantt Chart

This Gantt chart shows our two-semester plan for building a tool that turns construction blueprints into interactive 3D models, developed with feedback from our industry partner, Buildertrend. The fall semester covers research, prototyping, and a first working version, which ends in a presentation to Buildertrend. After the winter break, spring semester uses their feedback to add automatic floor alignment and roof generation. The project ends with a final review and presentation. The dashed red lines mark the key milestones.

1. **Prototype finalized** *(early November)*
   Finish the last version of the prototype. This closes out the research and prototyping phase and confirms the core approach before full implementation starts.
2. **Mid-project presentation to Buildertrend** *(mid-December)*
   Present the semester's work to Buildertrend: show the current features, explain the design decisions, and collect their feedback to guide spring development.
3. **Fall semester deliverable complete** *(end of fall semester)*
   Finish the final version of the fall project, including any quick fixes from the Buildertrend presentation. The first round of model testing and implementation ends here.
4. **Automatic floor alignment** *(early February)*
   Add automatic alignment so multi-story floor plans stack and connect correctly (walls, stairs, and structural elements line up between levels) with no manual adjustment.
5. **Roof generation from blueprints** *(early March)*
   Generate roofs automatically, pulling as much detail from the blueprint as possible, such as pitch, overhangs, and roofline shape, so the 3D model is complete and accurate.
6. **Final review with Buildertrend** *(mid-April)*
   Show Buildertrend the finished product and ask for final feedback to guide the last refinements.
7. **Final polish and presentation** *(end of April)*
   Act on the last round of feedback, polish the user experience, and give the final project presentation.

3.5 Risks and Risk Management/Mitigation

The following table summarizes the main project risks. Probabilities are preliminary estimates and will be reviewed as development progresses.

Task	Main risk	Probability	Mitigation plan
Input processing and computer vision	Differences in drawing styles or poor-quality scans cause missed walls, doors, or windows.	0.85	Test varied plans, improve preprocessing, and compare alternative detection methods. Allow manual correction of errors.
Data integration	Missing dimensions, inconsistent units, or schema mismatches produce incorrect model data.	0.80	Validate required values and units, version the shared schema, and test communication between components.
Client data handling	Confidential plans are exposed through public repositories or external services.	0.90	Keep plans in restricted storage, use synthetic examples publicly, and obtain approval before sending files to external services.

Table 1: Risk Management Table

3.6 Personnel Effort Requirements

Table 1 presents the estimated effort for each project task across both semesters. Person-hours represent the combined time contributed by all team members.

Task	Work included	Estimated person-hours
Requirements and research	Clarify client expectations, review existing approaches, and evaluate tools.	70
Input preparation and pre-processing	Load PDFs and images, identify relevant sheets, and normalize inputs as a second semester goal	100
Computer vision development	Prepare training data and develop detection of walls, stairs, doors, and windows. Then detect rooms throughout the floor.	180
Data post-processing and integration	Reconcile extracted information, validate shared JSON, and connect components.	120

User interface and visualization	Implement viewing controls, labels, manual corrections, and saving.	50
3D reconstruction	Generate walls, openings, floors, and supported roof geometry.	70
Multi-floor support and walkthrough	Position floors, connect levels, and implement movement and collision detection.	30
Testing and evaluation	Check accuracy, integration, performance, and usability of varying models, not including training times	50
Documentation and handoff	Prepare setup instructions, weekly reports, design documentation, and final demonstration	50
Total		**720**

Table 2: Estimated Time Table

3.7 Other Resource Requirements

The most important resource that our team needs and is required to complete the project with a quality model is a large dataset that covers many of the edge cases seen in real blueprints. This would give the model enough information to correctly identify any number of formats for doors, walls, stairs, and other niche home features and therefore, train successfully. Another requirement for the project will be having a license with YOLO in order to use the product for commercial purposes while not sharing all of the code publically. This was discussed with the client at length, and they gave us the go ahead to use it.

4 Design

4.1 Design Context

Our project helps homeowners, designers, and sales staff understand construction plans through interactive 3D models. People without experience reading blueprints may find it difficult to picture room layouts and building features. By making these plans easier to understand, our system aims to improve communication and help users identify misunderstandings before construction begins.

4.1.1 Broader Context

Our project is built mainly for homeowners, builders, and contractors who use Buildertrend to plan and discuss residential projects. Others who rely on the plans to make decisions, such as subcontractors and designers, are also affected. The table below covers the societal need our project addresses, which is making building designs easier to understand and communicate, along with the main considerations in each area.

Area	Description	Examples
Public health, safety, and welfare	Our 3D visualization helps homeowners and builders understand room layouts and communicate concerns before construction. Inaccurate geometry could mislead users, so missing or uncertain features should be clearly marked. With the use of customer blueprints, data privacy is also an important area to consider.	Flag uncertain wall dimensions or missing openings. Explain that the model is a visualization tool and does not replace professional review or building-code checks. Do not publicly post the blueprints used for training and testing.
Global, cultural, and social	Users have different levels of experience reading blueprints, and construction plans use different units and drawing conventions. Our system should make layouts understandable to non-technical users while preserving the intended design.	Provide simple navigation and readable room labels. Correctly interpret feet/inches and metric measurements without changing the building's proportions.
Environmental	Our project could help reduce construction waste by making building layouts easier to understand before construction begins. Identifying misunderstandings early may reduce changes that require materials to be removed or replaced.	A homeowner notices an unwanted wall layout in the 3D model and discusses a revision before construction, potentially avoiding demolition and wasted materials later.
Economic	Converting existing plans into 3D models could reduce manual modeling time and help Buildertrend users communicate designs more efficiently. These benefits must be balanced against development, licensing, and maintenance costs. It is also much more cost efficient to be able to quickly make a 3D model instead of hiring professional graphic designers.	Reduce time spent explaining layouts or recreating plans manually. Support operation without a dedicated GPU to limit hardware costs. Reduces costs from hiring graphic designers for modelling.

Table 3: Broader Context Table

4.1.2 Prior Work/Solutions

For this project we have found a few research papers that helped our understanding of the project and what implementations might be useful for it. The first research paper that was extremely helpful for the group was "Exploring the Utility and Challenges of AI Interpretation of Construction Blueprints" [1]. This research paper discussed several difficulties of this project, like how elements like stairs are often not consistent from blueprint to blueprint, as well as a lot of overlapping elements. It also really helped by giving a huge list of ways that different groups have tried to develop a model despite these things.

Another research paper that we used was "Two-Stage Floor Plan Recognition With a Network Employing Attention Mechanism" [2]. In this research paper they adopted an approach where they started off with taking in the basic geometry on the page and then afterwards tried to group the elements together into rooms. This is one approach that we are going to be using roughly. We are taking in the walls, doors, and other elements first, before having a separate model that makes the rooms. Additionally, there is the added benefit that the 2 models help verify one another. This approach has been very successful so far. The difference between our approaches is that we are using a YOLO model, which will help the model do better against unseen information but has the downside of requiring significantly more data.

The final research paper for this section that we used is "A complete system for the analysis of architectural drawings" [3]. In this research paper, they also do a similar approach to us while adding the 3D reconstruction element to it. One more similarity is the number of post-processing steps that are added, like straightening walls. Again, their approach struggled to work on images that they didn't focus on. With our approach, our model is much more likely to work on other approaches.

References:

[1] A. Shaout and M. Bryant, "Exploring the Utility and Challenges of AI Interpretation of Construction Blueprints," *International Journal of Advanced Networking and Applications*, vol. 16, no. 6, pp. 6627–6646, Jan. 2025, doi: 10.35444/IJANA.2025.16603.

[2] Y. Chen and C. Wang, "Two-Stage Floor Plan Recognition With a Network Employing Attention Mechanism," in *2024 IEEE 14th International Conference on Electronics Information and Emergency Communication (ICEIEC)*, Beijing, China, 2024, pp. 26–30, doi: 10.1109/ICEIEC61773.2024.10561665.

[3] P. Dosch, K. Tombre, C. Ah-Soon, and G. Masini, "A complete system for the analysis of architectural drawings," *International Journal on Document Analysis and Recognition*, vol. 3, no. 2, pp. 102–116, Dec. 2000, doi: 10.1007/PL00010901.

4.1.3 Technical Complexity

Our project system turns 2D construction floorplans into interactive 3D models through two connected subsystems. A computer vision pipeline that pulls structured building data out of plan images and a graphics rendering subsystem that lets users review, correct, and explore that data in 3D. Each subsystem has its own engineering principles, and together they handle requirements that existing floorplan-to-3D workflows mostly leave to manual work.

For the subsystem for computer vision, floor plans are hard to interpret automatically. Walls, doors, windows, dimension lines, and text labels often sit on top of each other, and drafting conventions vary a lot between designers and firms. Rather than relying on one general-purpose model, the pipeline splits the problem across several specialized models. These models may split up the work for finding the different variations of doors, windows, walls, and stairs. Putting the gathered information together from the diverse models and cross-checking them, we can create a more confident model that will recognize more edge cases. The raw model output by itself is rarely clean enough to use directly, so it goes through several post-processing steps, which we use computational geometry for. For example, wall segments that should connect are snapped together by clustering nearby endpoints within a distance tolerance and resolving their intersections. This produces a connected wall graph instead of a set of disconnected fragments. Training data adds another constraint. The client has provided only a small number of example plans, and public datasets such as CubiCasa5K don't match the client's drawing style and are not commercially available. To get reliable accuracy from this little data, the pipeline relies on mostly synthetic data generation.

As far as the graphics subsystem goes, we need a module that reads the pipeline's JSON output, validates it against a defined schema, and gives users a 2D editing view for fixing any detection errors before the 3D model is built. This human-in-the-loop step keeps the process fast while the user still controls the final result. The editor uses coordinate transformations and geometric constraint handling so that edits keep the building's structure intact. For multi-story buildings, each floor has to be aligned with the ones above and below it. The system does this automatically through stair alignment, but it still relies on users to verify it and step in manually when the automatic result needs adjusting. The aligned plans are then extruded into 3D meshes using polygon triangulation and standard transformation and projection matrices. Users can explore the model from an orbital view or a first-person walking view with collision detection. Because the application runs in the browser and is meant to work on typical hardware without a dedicated GPU, rendering performance is a core requirement.

4.2 Design Exploration

We have had several components of our design that needed to be worked out. Some examples of this are choosing which type of model we wanted to use for computer vision, what type of pipeline we wished to use, and how we wanted to display the 3d visualization.

4.2.1 Design Decisions

We chose YOLO for our computer vision model, and one of the primary reasons for this is that YOLO tends to perform well with future images that have not been seen yet. It does have the downside of not being able to work with very limited datasets. This requires us to have a significant amount of images for training.

For our pipeline, we chose to make it consist of multiple models rather than just one. This improves accuracy by specializing the tasks and gives our project a competitive edge over just throwing the plans into an existing highly intelligent chatbot.

As for the graphics engine, we decided on Three.js with WebGL because the nature of it working in-browser makes it inherently cross-compatible with many platforms, and Three.js provides helpful utilities over base WebGL. Our original prototype also used Three.js, so we have existing work to use if needed.

4.2.2 Ideation

We explored how to divide floor-plan interpretation tasks between computer vision models by first identifying five options that we had available to use. We chose these by starting off with the first option and then expanding based on what we learned during the testing process.

1. **Single model that uses boxes:** Detect walls, doors, windows, and stairs using one model.
2. **Single segmentation model:** Also detects walls, doors, windows, and stairs using one model.
3. **Grouped models:** Use one model for rooms and another for walls, stairs, doors, and windows.
4. **Sequential models:** Use a general model to identify regions, followed by specialized models to examine their details.
5. **Hybrid approach:** Extract available information directly from PDFs and use computer vision models for features requiring image interpretation.

4.2.3 Decision-Making and Trade-Offs

To then narrow down our options, we used a table to view it, modifying information as we learned and tested. To determine our type of model pipeline, we scored them on various metrics shown below and attempted to implement the one with the highest score. The scores are 1-10 per category, with 10 being the best. Accuracy was worth 1.5 times more than other sections. The reason we chose this method is because we want to have a situation where we are able to update our plan as we learn new information while still knowing exactly what we should be working towards at any given time.

	Single model that uses boxes	**Single Segmentation Model**	**Grouped Models**	**Sequential Models**	**Hybrid Approach**
Ease of implementation	10 (Very easy to implement)	8 (Somewhat easy to implement. Would have to work more on making sure most lines are straighter, though	6 (It was 4 at the start of this project. Would still be required to connect multiple trained models that don't have very similar things)	6 (It was 4 at the start of this project. Would still be required to connect multiple trained models that don't have very similar things)	7 (Wouldn't require the use of as much computer vision, but would require writing more cases for different labels)
Accuracy after initial testing: 1.5*Score	7 (Did surprisingly well ignoring stairs. Sometimes had random results)	3 (Struggled a lot more than expected. Felt much more likely to fail on new data)	8 (It mainly relied on one model for most information, but it did use one model specifically for the doors, which	9 (After initial testing, the 2 models were able to verify one another and had really good accuracy.)	Likely 8 (We haven't tested it out, so we don't know for sure. Would only work especially well on PDFs)

			helped.)		
Ease of Adaptability to new data sets	6 (Struggles with complex shapes a lot but works with objects that can be represented as a box)	9 (With segmentation, it can handle really complex shapes.)	7 (Could still struggle a bit with complex shapes, but not as much as the single model that uses boxes)	8 (Was able to work on more complex things due to the 2 models being able to validate one another)	6 (Only works better on PDFs; wouldn't work as well as other models for anything else.)
Final Score	**26.5**	**21.5**	**25**	**27.5**	**25**

Table 4. Decision-Making Table

Currently the approach we are working towards is now a sequential model using everything we have done so far. The biggest pro for this option is that we could change which models we use for specialized information as we test and find flaws.

4.3 Proposed Design

This section describes the design we are building to turn 2D floor plans into interactive 3D models. It begins with an overview of how a plan moves through the system, from upload to a model the user can orbit and walk through. It then breaks the system into its four subsystems (file management, computer vision, 3D reconstruction, and the user interface), shows how they connect through the shared JSON schema, and includes a block diagram. Next, it walks through what a user does with the system, step by step, and how that meets our requirements. It ends with the areas where the design is incomplete or at risk, and our plans to address them.

4.3.1 Overview

The program converts flat two-dimensional architecture floorplans into models in a three-dimensional space for viewing. It accomplishes this via a pipeline of multiple artificial intelligence models that are fine-tuned for different specialties and steps in the conversion process. The computer vision module accepts the floorplans as PDF files, puts the images through the pipeline, and outputs a JSON file containing model data. This JSON may then be both persisted to the file system and have its components modified in a 2D floorplan editor. Each floor is then aligned with one another and passed into the 3D renderer, which provides a visual representation of the model in both an orbital and walkthrough simulator.

4.3.2 Detailed Design and Visual(s)

The system turns 2D construction floor plans into interactive 3D building models that run in a web browser. It has four subsystems: file management, computer vision, 3D reconstruction, and the user interface. The work is split between the two subteams. The Computer Vision subteam's TypeScript pipeline reads a plan and writes the building out as JSON. The Graphics subteam's Three.js application turns that JSON into a 3D model the user can view, walk through, and edit. This JSON schema is the only contract between the two sides, so each side can be developed and tested independently. Every subsystem uses the same units and coordinate system. This semester covers single-floor plans, with multi-floor alignment and roof generation following in the spring.

- **File management** accepts an uploaded PDF, PNG/JPG, or existing JSON file. It takes in images for the vision pipeline or loads existing JSON schemas directly, and then exports finished models in the designated JSON schema format. This is all stored in a gallery, so the user can come back to it.
- **Computer vision** takes a page image and detects walls, doors, windows, stairs, and rooms using YOLO models. It uses PyMuPDF text extraction to read dimension labels, converts pixel coordinates into real-world measurements, and validates the result before writing it out as building JSON.
- **3D reconstruction** reads the building JSON into Three.js. It raises walls to their full height, cuts openings for doors and windows, builds floor surfaces, and places each floor at its elevation. The result is an in-memory model made of meshes, materials, and transforms.

- **User interface** renders that model with Three.js and WebGL. Users can orbit around the building, walk through it in first person with wall collision, correct any incorrectly detected elements, and export the edited model as a JSON file.

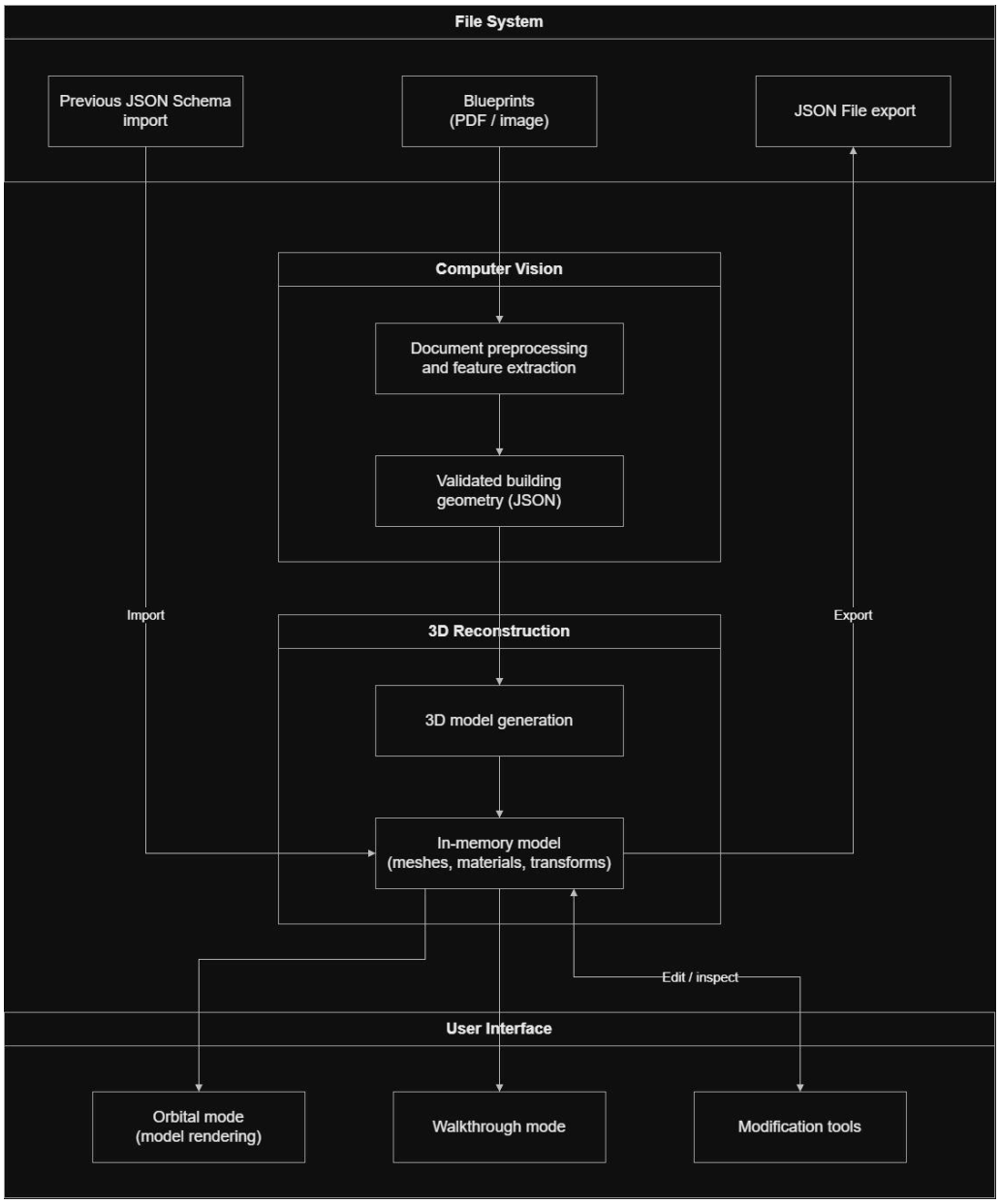


Figure 3. Block Diagram Overview

The computer vision subsystem is through scanned or image-only plans that go through YOLO models that detect the elements. Post-processing then snaps wall endpoints together and attaches each door and window to its wall. A dimension or a scale is read either from units in the image or by guessing based on door size. The output is checked for missing dimensions and disconnected walls before it is written as JSON through the multi-stage postprocessing step. Finally, the graphics side loads the JSON and builds one in-memory model. The user's edits change that same model, so the orbit and walkthrough view change in real time. The system needs no dedicated GPU, and client plans never leave team-controlled storage used solely for training.

As for the file system, users can upload blueprints as PDFs or images, or import existing JSON structures. PDF pages are converted into images for processing. Existing JSON files are loaded directly into the in-memory model. After viewing or editing, users can export the updated model as a JSON schema once again.

The 3D reconstruction subsystem generates walls using their endpoints, thickness, and height. Door and window positions define openings within those walls. Closed floor boundaries are converted into floor meshes, and multiple floors are placed at their specified elevations. The resulting in-memory model contains meshes, materials, and transforms describing each object's geometry, appearance, position, rotation, and scale. Dimensions unavailable from the blueprint require user input or documented defaults.

The final subsystem is the user interface. There is an orbital viewing mode that allows users to rotate, pan, and zoom around the model. There is also a walkthrough mode providing an interior view with movement controls and collision checks to prevent movement through walls. There are also easily usable modification tools in the pipeline, which allow the users to select objects and change supported properties, such as position and dimensions. These changes update the shared in-memory model, keeping both viewing modes and exported files consistent.

4.3.3 Functionality

The system is meant to be used by a non-technical person, such as a homeowner or a salesperson, with no training and in no more than three main steps. This semester it handles single-floor plans.

1. **Upload a plan.** The user uploads a floor plan as a PDF or as a PNG or JPG image. The system checks the file first. If the file is not a plan it can read, it stops with a clear error instead of producing a broken model.
2. **The system reads the plan.** For scanned images, the computer vision models detect walls, doors, windows, stairs, rooms, roofing, etc. Post-processing cleans up the results and writes them as JSON that follows the shared schema.
3. **View the 3D model.** The graphics side builds walls, openings, floors, and roofs from the JSON and shows the model in the browser with orbit, pan, and zoom controls. Room names appear as labels the user can turn on and off.
4. **Check and correct.** If the system was unsure about an element or could not extract part of the plan, the element is visually marked as low confidence, so the user knows what to double-check. The user can fix mistakes, such as moving a wall or changing a door width, and the model updates.
5. **Walk through the building.** The user switches to a first-person mode and moves through the building with the keyboard and mouse. Collision with walls keeps the user inside the layout.

A salesperson would use this in a meeting by uploading the customer's plan and walking the customer through the rooms. A homeowner could do the same on their own to check whether a layout matches what they pictured before construction begins.

4.3.4 Areas of Concern and Development

The design is early and meets most requirements only in part. The prototype turns a floor plan image into a 3D model with a walk mode, and the shared JSON schema is in use. The extraction script matched door and window counts on the first sample plan but did not work on the other three. We will need to add some more sub-labels for those and improve the models through more training samples. Reconciling elevations, manual correction, and low-confidence marking in the 3D view are not built yet. The prototype is not very accurate, so it would not meet the client's and user's needs yet. If we are able to create the design that we are planning, the user needs will be fulfilled as described. Our models that are not yet a part of the prototype have so far been fairly successful minus some needed post-processing steps. We have two main concerns with the current state of our design:

- The model misses some walls and adds false ones where furniture interferes with lines. We have found more public datasets, but they may not look like Buildertrend's real plans, so we need more sample plans from the client. The schema and extraction script were also built from one blueprint and do not yet cover some other labels.
- Most plans contain no texture or material information, so there is nothing to extract. The client has shown little interest in textures, so we plan to use simple neutral materials unless the client says otherwise.

A solution to address these would either create more accurate synthetic plans, or get more examples from Buildertrend. We will finish and test the current model, build the post-processing steps, and revise the schema for the other sample plans. We will also look for more ways to improve known shortcomings with the models by adding more steps in the pipeline.

Questions for clients, TAs & advisors

- Can Buildertrend provide more sample plans, including potentially handwritten scanned ones?
- What accuracy would Buildertrend consider good enough?

4.4 Technology Considerations

YOLO. Some strengths would include handling plans without a text layer and being quick to run once trained. It is also strong in a series of different models. Weaknesses might include the fact that training takes about 1 hour for every 6 epochs for some models, and boxes do not represent diagonal walls well using a basic model. A vector polygon model was tried and performed poorly on its own. An alternative approach would include trying out Otsu, which reads the pixels directly instead of calculating vectors or boxes.

JSON schema: The strengths with using a JSON schema are that both subteams develop and test independently against one contract, so we never need to rely on each other's direct implementation. On the other hand, the schema was built from one blueprint, so porches, multi-plane roofs, and stepped floors are not covered yet, making real blueprints weak. A possible alternative might be a looser format, such as a list, which might let us change easier over time but would break validation every time until we update it.

Three.js. The strength with using Three.js is that it is a well-optimized and documented library that is cross-platform. It is also a much simpler approach than using something like Blender or a game engine. A weakness is that running a graphics engine in the browser will never be as optimal as a standalone program. For our use case, the compatibility is worth it. An alternative approach would be using base WebGL or WebGPU. We also could think about using such game engines but would prefer to keep everything localized.

4.5 Design Analysis

Overall, the main path of the design in 4.3 works, but the design is not complete. We have built and run the riskiest parts from start to finish. We got usable JSON structures out of a plan image and passed it across the JSON contract. The parts that depend on them are only partly done. These are accuracy on varied plans, the correction tools, and roof and elevation data. Currently the pipeline goes from an uploaded floor image to a 3D model in the browser. Two YOLO models do the detection. One finds walls, doors, windows, and stairs as boxes. A separate model outlines rooms. In post-processing, each model checks the other. Room outlines snap to wall junctions, and walls that border a detected room get higher confidence. This is the two-model approach that scored highest in Table 4, and our testing so far supports that choice. On the CubiCasa5K test split, the room model scored 0.88 at finding rooms and was 87% accurate on room type. The output validates against our JSON schema, and the Three.js viewer builds walls, openings, and floors from it. It then lets the user view the model in an orbit and walkthrough mode, making this part complete. The JSON schema is the only link between the two subteams, so either the detector or the renderer can be replaced without changing the other side. Training needs a GPU, but the trained models can run on a CPU, so the first resource requirement (RR-1) still holds for end users.

Currently there are still some issues in regard to accuracy on Buildertrend plans. The models were trained mostly on public and synthetic data that are different from the files that were given by Buildertrend. The biggest thing we need to work on for the rest of the semester is improving the accuracy of our model. Additionally, diagonal walls still don't work well. The rotated (theta) model we are training is meant to fix this. As for feasibility, the design is still feasible within two semesters. There are still some concerns regarding insufficient data sets, however, a huge chunk of the remaining work with the model involves refining the current architecture rather than replacing it.

For the rest of the semester, we plan to help our concerns by building a held-out test set of hand-labeled Buildertrend floors that is never used for training. Additionally, in the near future, we plan to finish the theta model and compare it with the

current box model on the held-out set. We would also like to add low-confidence markings this semester. Finally, in the second semester we plan on adding roofing and elevation details to the project.

5 Testing

Testing is an **extremely** important component of most projects, whether it involves a circuit, a process, power system, or software.

The testing plan should connect the requirements and the design to the adopted test strategy and instruments. In this overarching introduction, give an overview of the testing strategy and your team's overall testing philosophy. Emphasize any unique challenges to testing for your system/design.

In the sections below, describe specific methods for testing. You may include additional types of testing, if applicable to your design. If a particular type of testing is not applicable to your project, you must justify why you are not including it.

When writing your testing planning consider a few guidelines:

- Is our testing plan unique to our project? (It should be)
- Are you testing related to all requirements? For requirements you're not testing (e.g., cost related requirements) can you justify their exclusion?
- Is your testing plan comprehensive?
- When should you be testing? (In most cases, it's early and often, not at the end of the project)

5.1 Unit Testing

What units are being tested? How? Tools?

5.2 Interface Testing

What are the interfaces in your design? Discuss how the composition of two or more units (interfaces) are being tested. Tools?

5.3 Integration Testing

What are the critical integration paths in your design? Justification for criticality may come from your requirements. How will they be tested? Tools?

5.4 System Testing

Describe system level testing strategy. What set of unit tests, interface tests, and integration tests suffice for system level testing? This should be closely tied to the requirements. Tools?

5.5 Regression Testing

How are you ensuring that any new additions do not break the old functionality? What implemented critical features do you need to ensure do not break? Is it driven by requirements? Tools?

5.6 Acceptance Testing

How will you demonstrate that the design requirements, both functional and non-functional, are being met? How would you involve your client in the acceptance testing?

5.7 Security Testing (if applicable)

How will you demonstrate that the security requirements, if any, are being met? What data and/or resources are being protected? What security testing principles, methods, and/or tools may be applied?

5.8 User Testing

Describe how the team will include real users to interact with the solution/product. Your user testing plan should evaluate your solution/product's usability and overall user experience by gathering user feedback through observations, surveys, etc. User feedback allows the team to identify areas for improvement and ensure the product meets user needs.

5.9 Testing Results

What are the results of your testing thus far? Include any numerical, graphical, or qualitative testing results here. How do they demonstrate compliance with the requirements or addressing user needs? Use a summary narrative to discuss what you've learned and what next steps need to be taken.

6 Implementation

Describe any (preliminary) implementations of your design thus far. Support any general, descriptive text with relevant images. If your project has inseparable activities between design and implementation, you can list them either in the Design section or this section.

7 Ethics and Professional Responsibility

Use this section to describe your considerations of engineering ethics and professional responsibility. Most importantly how are you defining engineering ethics and professional responsibility in the context of your project and what steps are you taken to ensure ethical and responsible conduct. Each section references one type of ethical/professional responsibility considerations. You may also use this introductory section to note any overarching ethical philosophy among your team.

7.1 Areas of Professional Responsibility/Codes of Ethics

This discussion is with respect to the paper by J. McCormack and colleagues titled "Contextualizing Professionalism in Capstone Projects Using the IDEALS Professional Responsibility Assessment", *International Journal of Engineering Education* Vol. 28, No. 2, pp. 416–424, 2012

Pick one of IEEE, ACM, or SE code of ethics (all linked in class slides). Create a table, like Table 1 in the McCormack et al. (2012, pg. 418)) paper, with the following columns representing: Area of Responsibility (from the paper), Definition (in your own words), Relevant Item from Code of Ethics (from the Code of Ethics you selected, and description of how your team has interacted with that area of professional responsibility or adhered to that code during your project thus far.

In text below the table, describe one area in which your team is performing well. Describe what your team is doing and how that signifies strong performance. Also describe one area in which your team needs to improve. Describe what your team is currently doing and what it should do in the future to improve.

7.2 Four Principles

Create a table with rows for each broader context area (see Section 4.1.1) and columns for each of the four principles (beneficence, nonmalificence, respect for autonomy, and justice; see Beauchamp, 2007). Within the table, identify at least one way each of the four principles applies to each of the broader context areas. Some principle-broader context connections might be more prominent than others, but you should be able to identify something for each table cell. Note: Your design may end up negative or neutral in some cell. For example, your product might perform poorly in environment-nonmaleficence because it utilizes natural resources without a positive/mitigating effect.

Below the table, note one broader context-principle pair that is important to your project. Briefly describe the benefit in that area you are working towards and how you will ensure it. Also note one broader context-principle pair in which your project/end design is or will be lacking. Describe either (a) how this negative is overcome by other positives in other areas of the project/design or (b) what your team must do to improve in this area.

7.3 Virtues

List and define at least three virtues that are important to your team. Describe what you will do or have done as a team to support these virtues among all team members.

Each team member should also answer the following:

- Identify one virtue you have demonstrated in your senior design work thus far? (Individual)
    - Why is it important to you?
    - How have you demonstrated it?
- Identify one virtue that is important to you that you have not demonstrated in your senior design work thus far? (Individual)
    - Why is it important to you?
    - What might you do to demonstrate that virtue?

8 Closing Material

8.1 Conclusion

Summarize the work you have done so far. Briefly re-iterate your goals. Then, re-iterate the best plan of action (or solution) to achieving your goals. What constrained you from achieving these goals (if something did)? What could be done differently in a future design/implementation iteration to achieve these goals?

8.2 References

List technical references and related work / market survey references. Do professional citation style (ex. IEEE). See link:

https://ieee-dataport.org/sites/default/files/analysis/27/IEEE%20Citation%20Guidelines.pdf

8.3 Appendices

Any additional information that would be helpful to the evaluation of your design document.

If you have any large graphs, tables, or similar data that does not directly pertain to the problem but helps support it, include it here. This would also be a good area to include hardware/software manuals used. May include CAD files, circuit schematics/layout, PCB testing issues, software bugs, etc.

9 Team

Complete each section as completely and concisely as possible. We strongly recommend using tables or bulleted lists when applicable.

9.1 Team Members

9.2 Required Skill Sets for Your Project

If feasible – tie them to the requirements.

9.3 Skill Sets Covered by the Team

For each skill, state who covered it.

9.4 Project Management Style Adopted by the team

Typically, waterfall, agile, or hybrid for project management.

9.5 Initial Project Management Roles

Enumerate which team member plays what role.

9.6 Team Contract

Team Members:

1) _____ 2) _____

3) _____ 4) _____

5) _____ 6) _____

7) _____ 8) _____

Team Procedures

1) Day, time, and location (face-to-face or virtual) for regular team meetings:
2) Preferred method of communication updates, reminders, issues, and scheduling (e.g., e-mail, phone, app, face-to-face):
3) Decision-making policy (e.g., consensus, majority vote):
4) Procedures for record keeping (i.e., who will keep meeting minutes, how will minutes be shared/archived):

Participation Expectations

1. Expected individual attendance, punctuality, and participation at all team meetings:
2. Expected level of responsibility for fulfilling team assignments, timelines, and deadlines:
3. Expected level of communication with other team members:
4. Expected level of commitment to team decisions and tasks:

Leadership

1. Leadership roles for each team member (e.g., team organization, client interaction, individual component design, testing, etc.):
2. Strategies for supporting and guiding the work of all team members:
3. Strategies for recognizing the contributions of all team members:

Collaboration

1. Describe the skills, expertise, and unique perspectives each team member brings to the team.
2. Strategies for encouraging and support contributions and ideas from all team members:

3. Procedures for identifying and resolving collaboration or contribution issues (e.g., how will a team member inform the team that the team environment is impeding their opportunity or ability to contribute?)

Goal-Setting, Planning, and Execution

1. Team goals for this semester:
2. Strategies for planning and assigning individual and team work:
3. Strategies for keeping on task:

Consequences for Not Adhering to Team Contract

1. How will you handle infractions of any of the obligations of this team contract?
2. What will your team do if the infractions continue?

****

a. I participated in formulating the standards, roles, and procedures as stated in this contract.
b. I understand that I am obligated to abide by these terms and conditions.
c. I understand that if I do not abide by these terms and conditions, I will suffer the consequences as stated in this contract.

1) _____ DATE _____

2) _____ DATE _____

3) _____ DATE _____

4) _____ DATE _____

5) _____ DATE _____

6) _____ DATE _____

7) _____ DATE _____

8) _____ DATE _____